

Introduction to MLOps

Vladislav Goncharenko



fall 2024

Vladislav Goncharenko

- Author of machine learning courses and Masters program at MIPT
- Associate professor at MSU, HSE, Harbour.space
- ML researcher (MIPT)
- Team lead of video ranking team at Dzen (yandex.ru)
- Ex-team lead of perception team at self-driving trucks (Evocargo)
- Open source fan



girafe
ai



Дзен

Preface

girafe
ai

00

Prerequisites

- UNIX command line basics
 - git basics
- Python programming experience
 - web services is a plus
- Basic machine learning knowledge
 - sklearn
 - pytorch

Disclaimer

- We discuss broad way to do things
- However there are plenty of ways
- We prefer ways suitable for most ML engineers
 - meaning not very deep software engineering skills
- Python is main tool for this course
 - MLOps exists in other stacks too, but for ML it's mostly Python

Your part

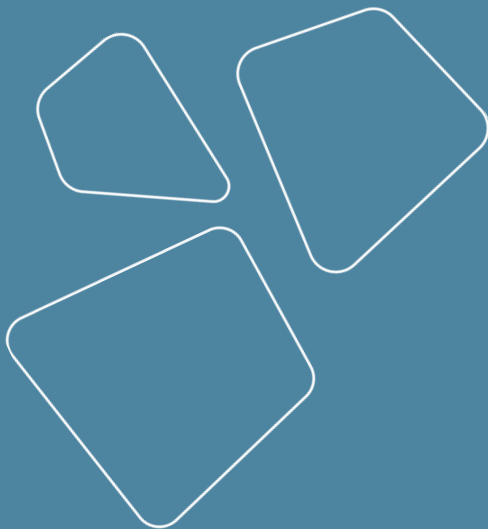
1. Knowledge level of each student is different
2. We adjust materials based on your knowledge
3. Your knowledge is estimated by feedback

More you ask - more you get from this course!

I can give you a way but you need to make an effort to follow it

Outline

- Motivation
- What is MLOps?
- Subtasks
 - course structure
- Stacks overview
- Homeworks outline



Motivation for MLOps

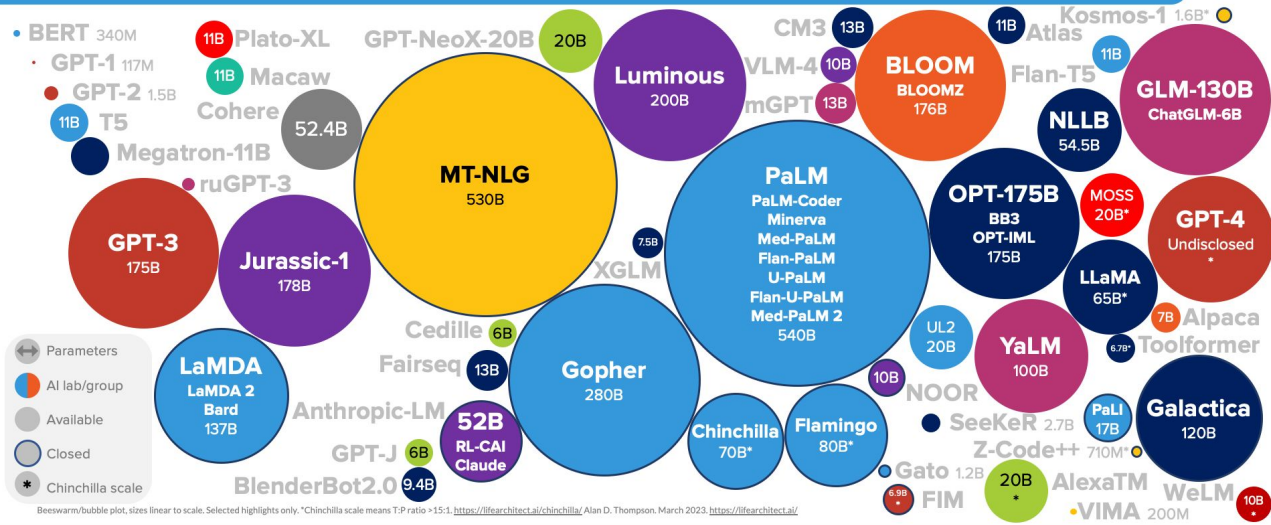
girafe
ai

01

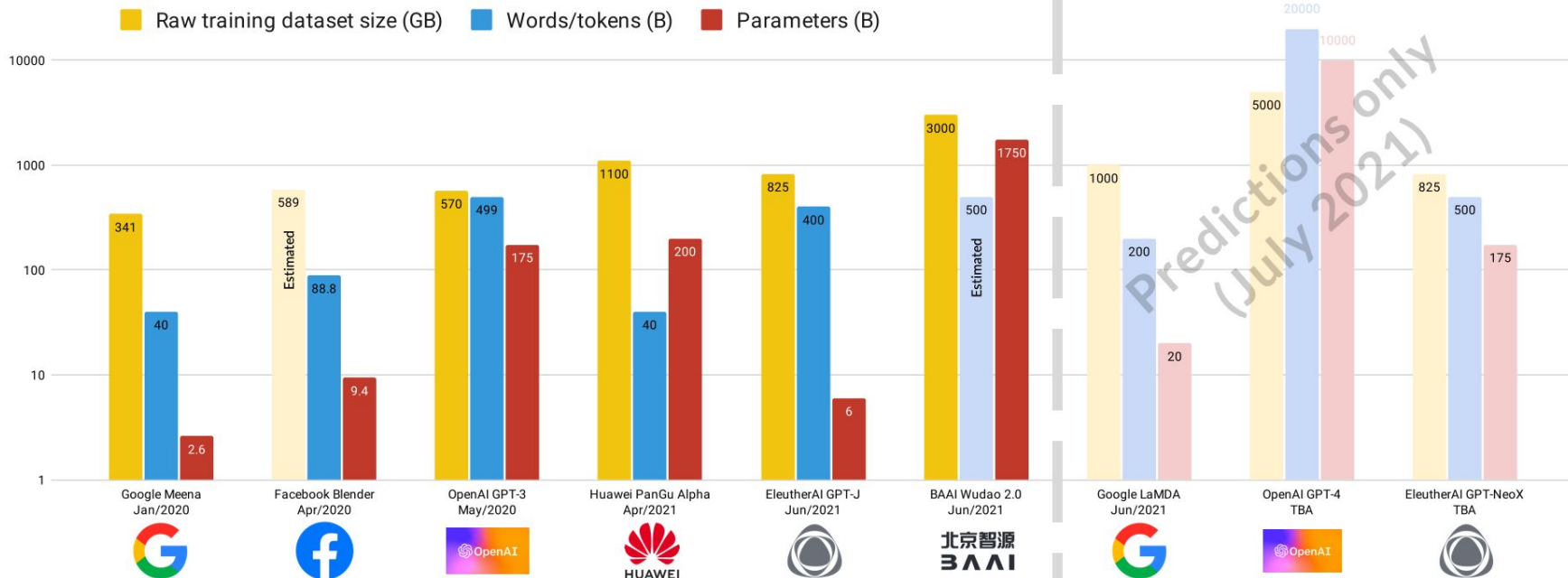
Why MLOps is needed?

- Resources for ML are growing
 - models training (gpu)
 - data mining (cpu, disks)
 - labeling (people)

LANGUAGE MODEL SIZES TO MAR/2023



LANGUAGE MODEL SIZES & PREDICTIONS (JUL/2021)



Alan D. Thompson, July 2021.
<https://lilearhitect.com.au/al/>

Why MLOps is needed?

- Resources for ML are growing
 - data mining (cpu)
 - labeling (people)
 - models training (gpu)
- Results reproduction
 - not only for industry, but for academia as well

Why Most Published Research Findings Are False

🌐 1 language ▼

Article [Talk](#)

[Read](#) [Edit](#) [View history](#) [Tools](#) ▼

From Wikipedia, the free encyclopedia

"Why Most Published Research Findings Are False" is a paper by Ioannidis et al. published in the [Stanford School of Medicine](#) [metascience](#).

In the paper, Ioannidis argues that many published research findings contain results that cannot be replicated. He determines whether scientific findings are based on formalized calculations ("P-values") or on untested assumptions about the world, which indicates that most published research findings are false.

COMMUNICATIONS OF THE ACM

[HOME](#)

[CURRENT ISSUE](#)

[NEWS](#)

[BLOGS](#)

[OPINION](#)

[RESEARCH](#)

[PRACTICE](#)

[CAREERS](#)

[ARCHIVE](#)

[VIDEOS](#)

[Home](#) / [Magazine Archive](#) / [August 2020 \(Vol. 63, No. 8\)](#) / [Threats of a Replication Crisis in Empirical Computer Science](#) / [Abstract](#)

REVIEW ARTICLES

Threats of a Replication Crisis in Empirical Computer Science

By Andy Cockburn, Pierre Dragicevic, Lonni Besançon, Carl Gutwin

Communications of the ACM, August 2020, Vol. 63, No. 8, Pages 70-79

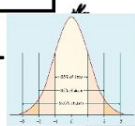
Why MLOps is needed?

- Resources for ML are growing
 - data mining (cpu)
 - labeling (people)
 - models training (gpu)
- Results reproduction
 - not only for industry, but for academia as well
- Models delivery
 - reduce time to market
 - reduce routine
 - testing and reliability
- Competence decomposition
 - deeper division of labor

Why do ML specialists need MLOps?

The Virgin Statistics

Entire skillset falls to pieces the moment they work with an uncommon distribution



Wears suits and cologne to seem more professional

Obsessed with plotting. Claims it helps people understand, but it only confuses them more.



Convolved, ineffective methods get rebuffed at every work meeting.

Obsessed with theory. Can't do anything without stealing the methods of smarter, dead men.



Secretly resentful, wishes his field was more popular.

Constantly begging superiors for more data.

"I k...know the results seem weird, b-but the p-value is considered significant..."

Decisions:	The Null Hypothesis is True	False
Accept H_0	(1- α) Confidence Level	β
Reject H_0	α	(1- β) Power of Test

Checks his results.

Knows *maybe* three methods, all requiring impossible assumptions.

Simple Linear Regression Model

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$

Assumptions: Random, Independent, Homoscedastic, Normality

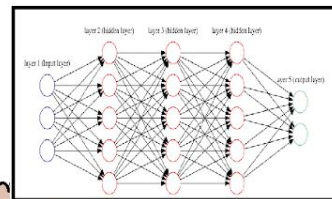
The Chad Machine-Learning

Regularizes his data until it gives the result he wants.



Has literally never heard the word "distribution" in his life

Fastest growing and most in demand discipline in his country.



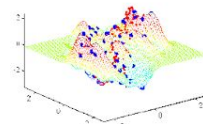
Innovates daily, advancing his field while solving real problems.

Doesn't try to explain methods, simply gives results, and is respected for it.

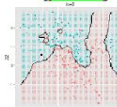
Wears sweats and muscle shirts to work, no one can stop him because he's too valuable.

Has so much data, often has to throw it away.

Let's his subordinates waste their days making his plots, would be a waste of his talent.



Refuses to use the same method more than once, each problem is unique and requires its own touch.



Implements methods into production without testing.

Why do ML specialists need MLOps?

because no one will do it for you!!!

For deeper dive I suggest reviewing
[system design primer](#) - cookbook for modern web based systems

What is MLOps?

girafe
ai

02

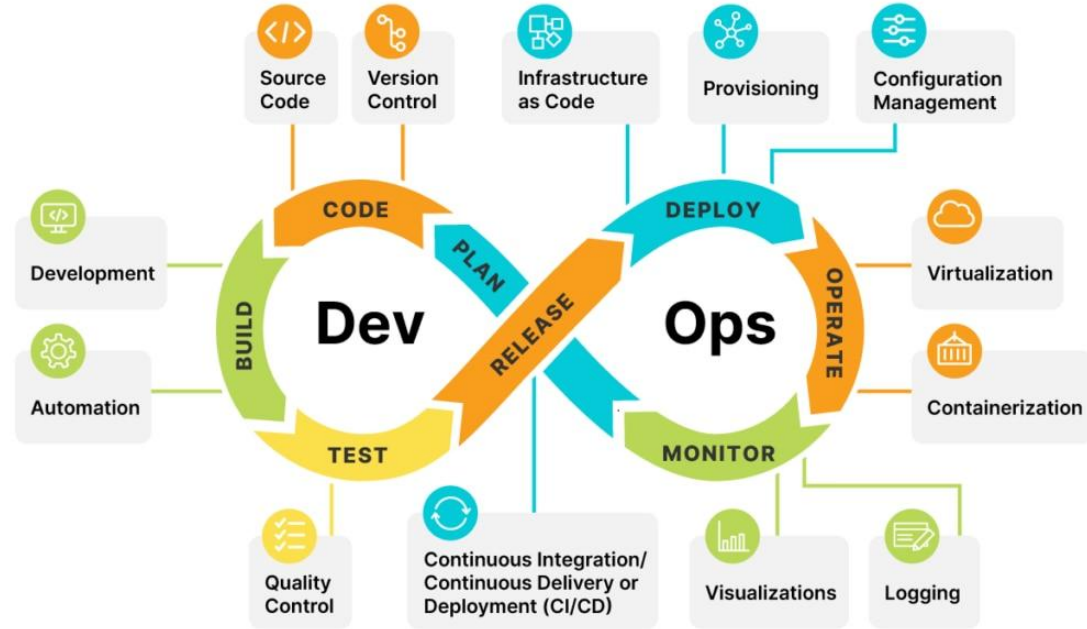
DevOps definition

DevOps is a methodology in the software development and IT industry. Used as a set of practices and tools.

DevOps integrates and automates the work of software development (Dev) and IT operations (Ops) as a means for improving and shortening the systems development life cycle

[Well-known common knowledge site](#)

[Great talk on DevOps practices](#)

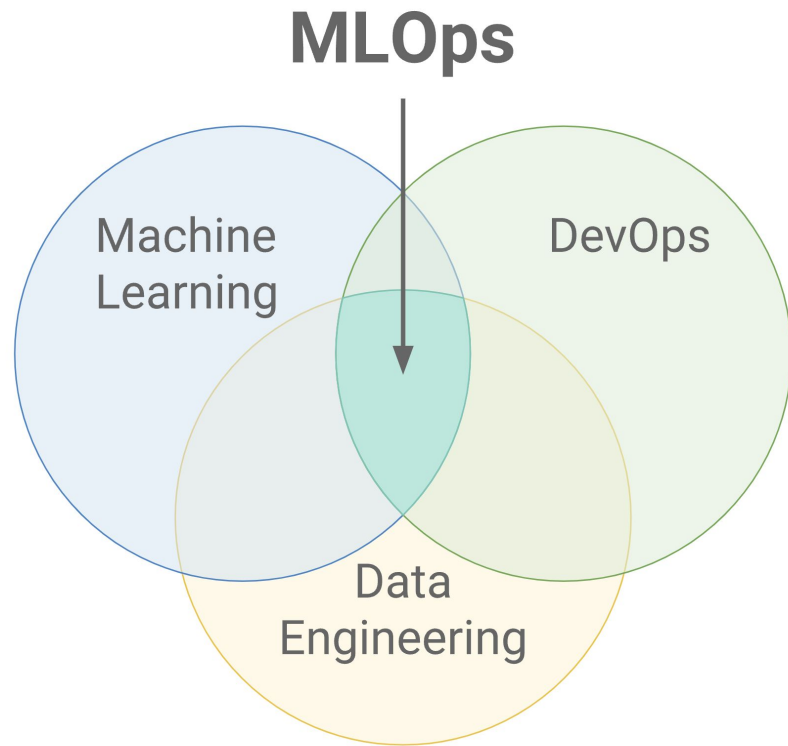


MLOps definition

MLOps is a paradigm that aims to deploy and maintain machine learning models in production reliably and efficiently.

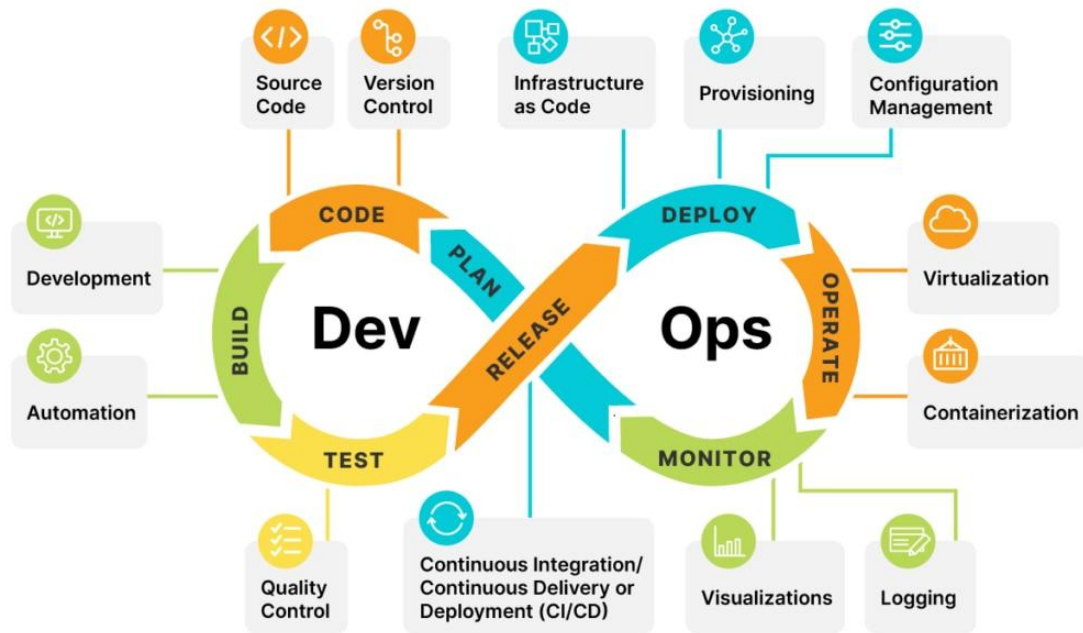
MLOps seeks to increase automation and improve the quality of production models, while also focusing on business and regulatory requirements

[Well-known common knowledge site](#)



Relations to other spheres

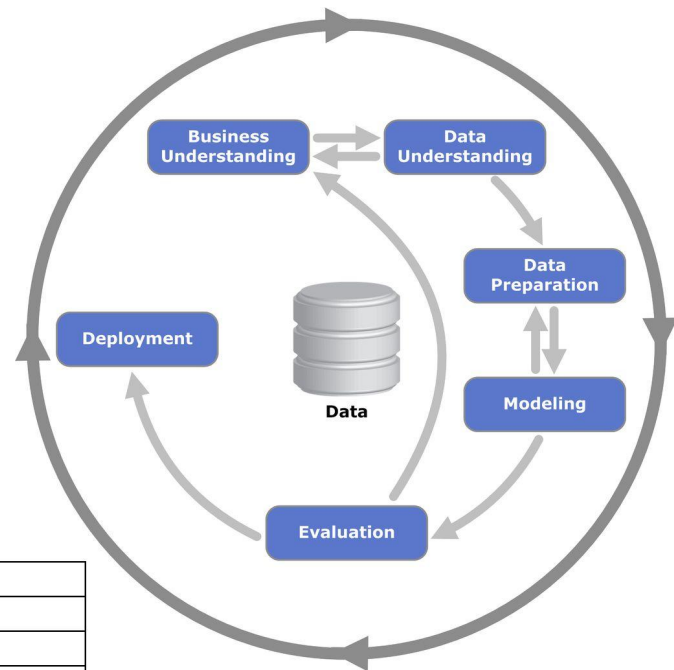
- Management
- Software development
- SQL
- Analytics
- Data bases
- Machine Learning



CRISP DM

In the field of Data Science is used. **Cross-industry standard process for data mining** ([CRISP DM](#)) is a process standard for projects using predictive analytics, proposed in 1999. [It was used by half of the companies](#) engaged in Data Mining at the beginning of the century.

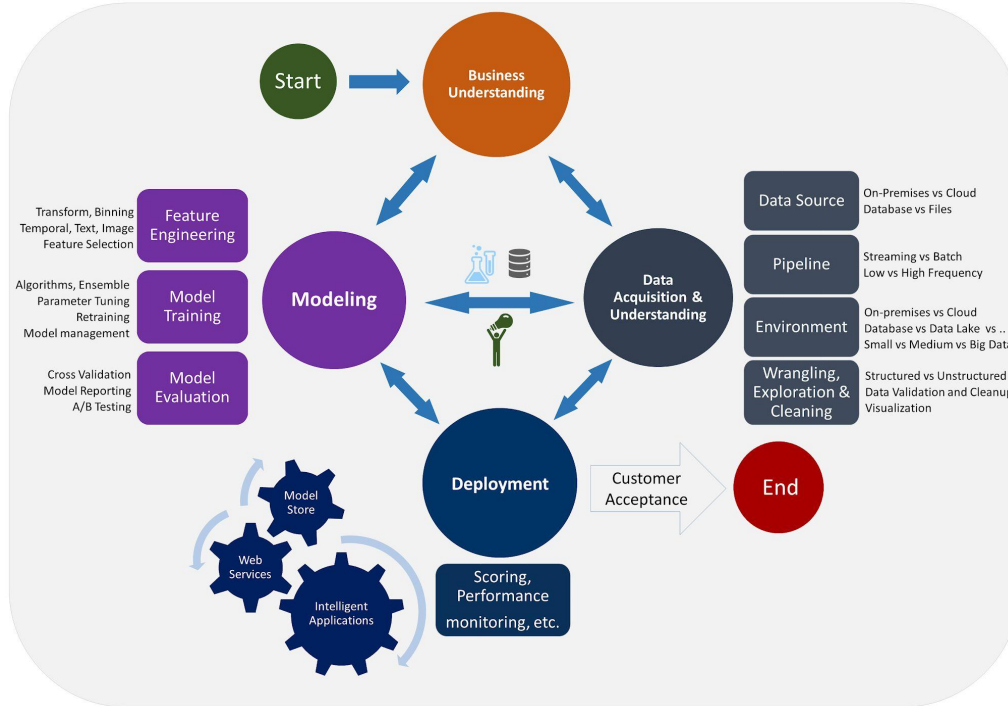
[Read more](#)



Poll Years	2002	2004	2007	2014
CRISP-DM	51%	42%	42%	43%
SEMMA	12%	10%	13%	8.5%
KDD process			7%	7.5%
My organization's	7%	6%	5%	3.5%
My own	23%	28%	19%	27.5%
Other (incl. domain specific)	4%	6%	9% (5%)	10% (2%)
None	4%	7%	5%	0%

Team Data Science Process

Data Science Lifecycle

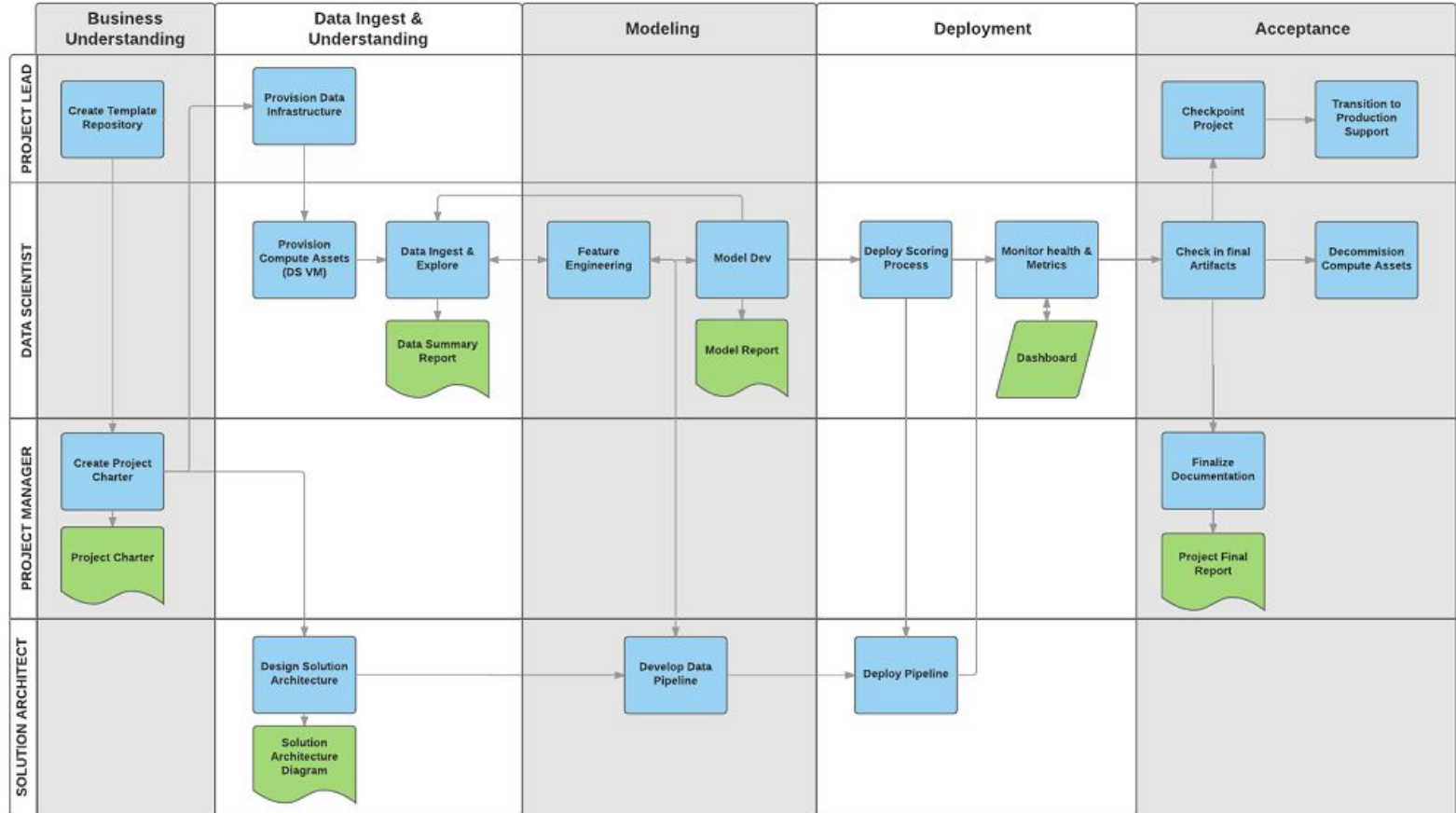


Modern standard from Microsoft

<https://learn.microsoft.com/en-us/azure/architecture/data-science-process/overview>

It is distinguished by the allocation of roles and a more detailed study of the stages of the cycle

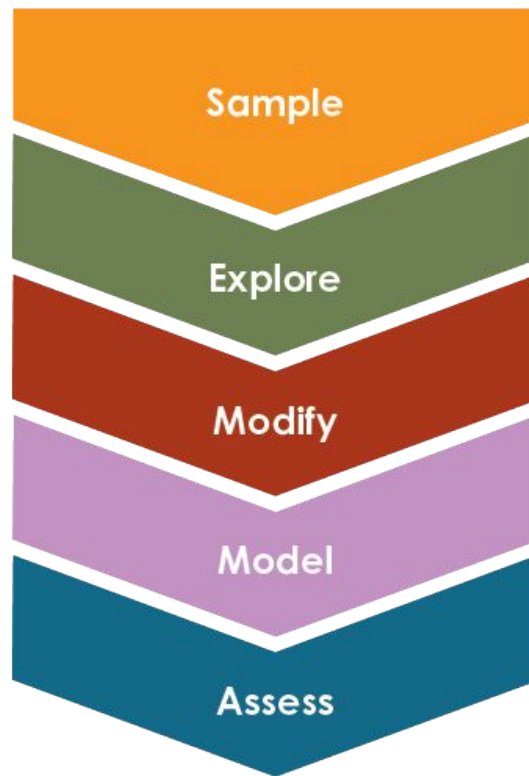
TDSP: team roles



Other approaches

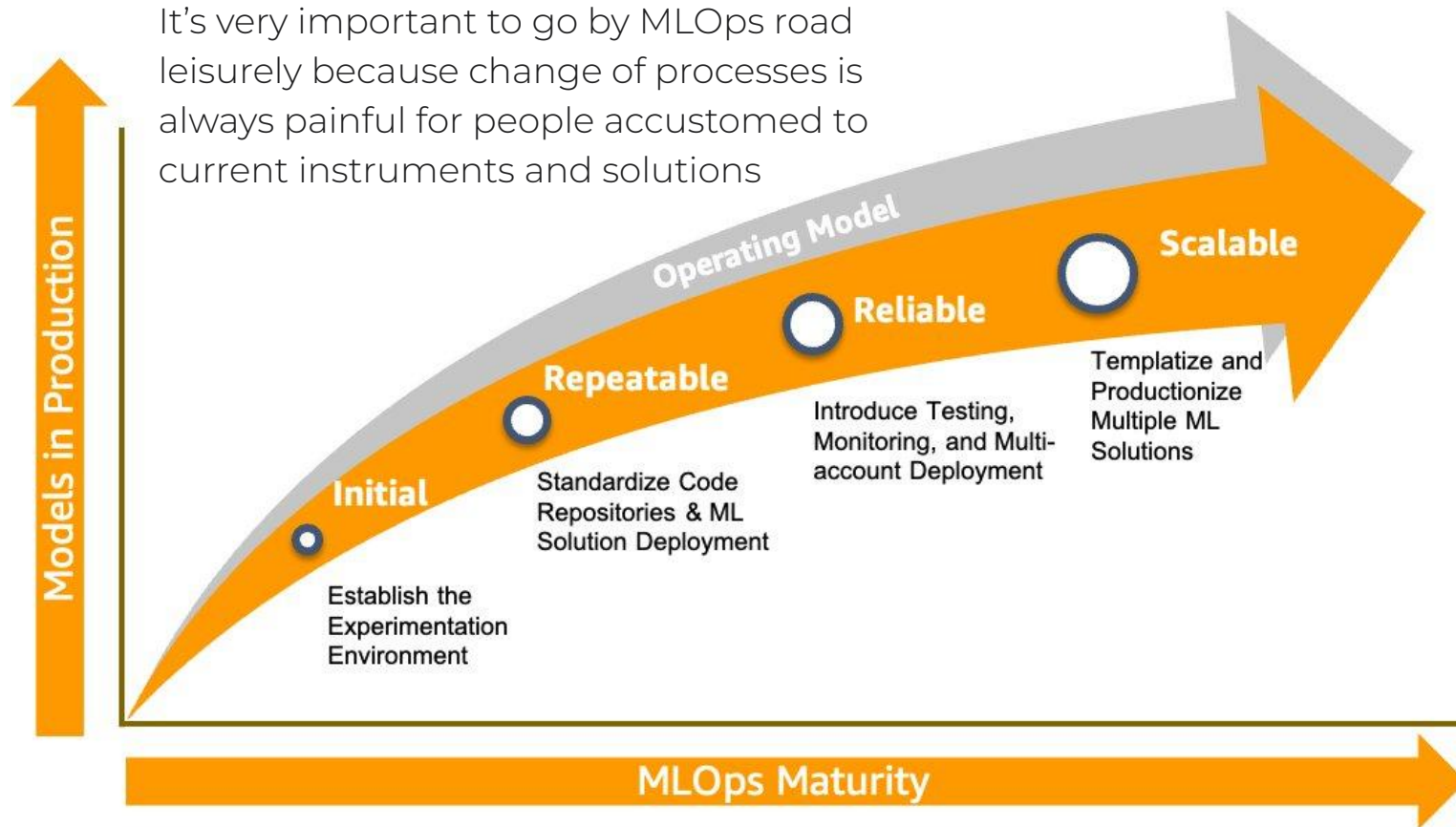
1. [SEMMA](#)
[read more](#)
2. [KDD Process](#)
3. [Waterfall](#)
4. etc...

In fact these are topics for IT management course



MLOps gradual introduction steps

It's very important to go by MLOps road leisurely because change of processes is always painful for people accustomed to current instruments and solutions



MLOps subtasks

Course structure

girafe
ai

03

MLOps subtasks

- Contemporary solutions knowledge
- Software engineering
 - Python infrastructure
- Virtualization
 - Docker
- Code storage and management
 - Git
 - CI & CD
 - code quality
 - testing (auto tests, test stands)
 - documentation generation
- Data storage and management
 - Data Acquisition
 - Data Engineering
 - AB tests

MLOps subtasks

- Contemporary solutions knowledge
- Software engineering
- Virtualization
- Code storage and management
- Data storage and management
- Labeling and crowdsourcing
- Experiments tracking
 - production monitoring
- Computational resources
 - GPU
- Model storages and deployment preparation
- Efficient inference
- Embedding storage optimization and vector databases

MLOps stacks overview

girafe
ai

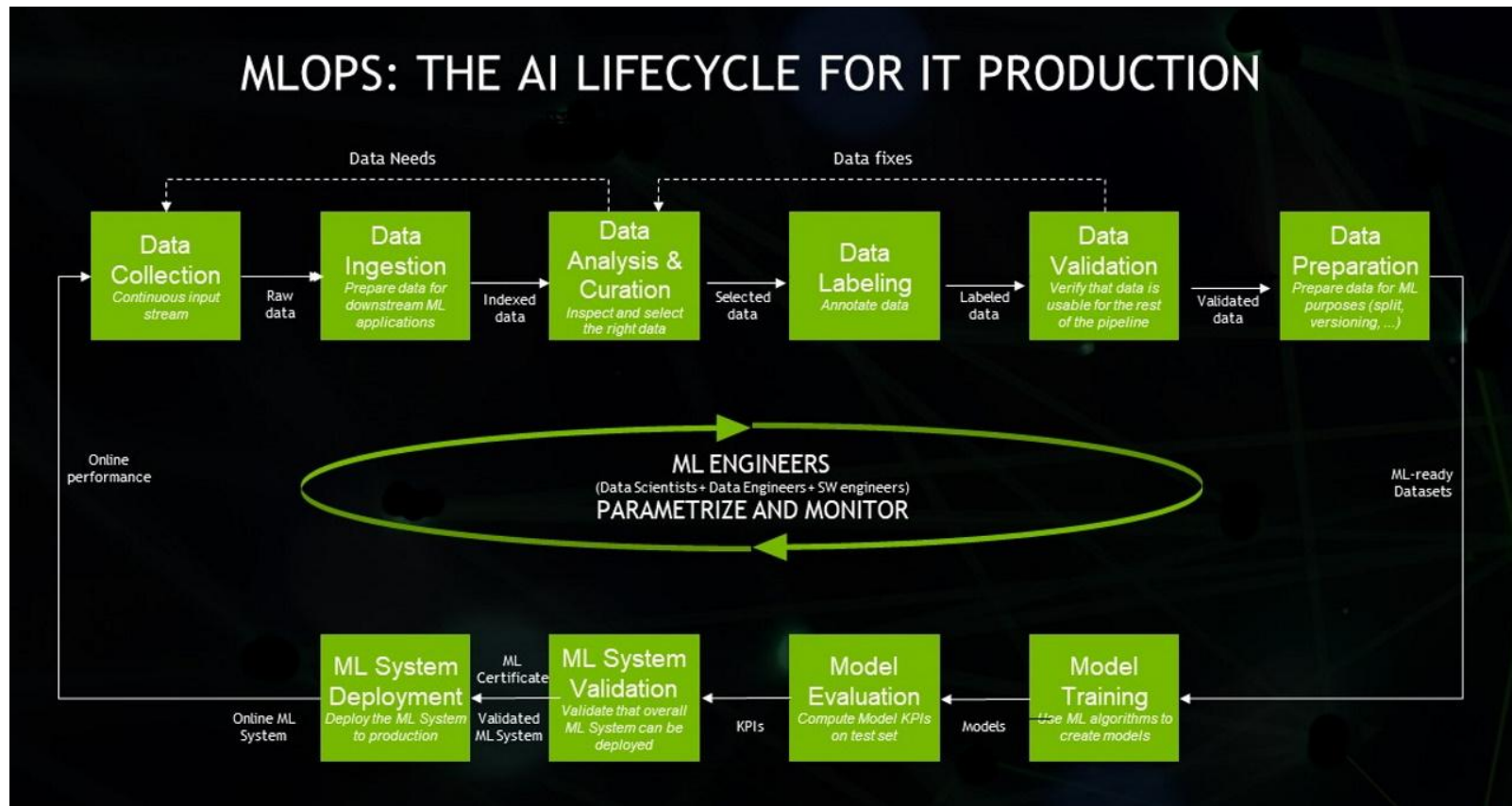
04

MLOps at big tech

Big companies have their stack to organize models training and publishing pipelines

- [Amazon SageMaker](#)
- [Google Cloud](#)
- [Nvidia](#)
- [Microsoft Azure](#)
- Yandex: [YTSaurus](#) + Nirvana + [Toloka](#)
- [Databricks](#)
- [Seldon](#)

Generally variance of instruments is huge over different companies (from only bare metal ssh machines to very solid custom solutions)



Not only big tech

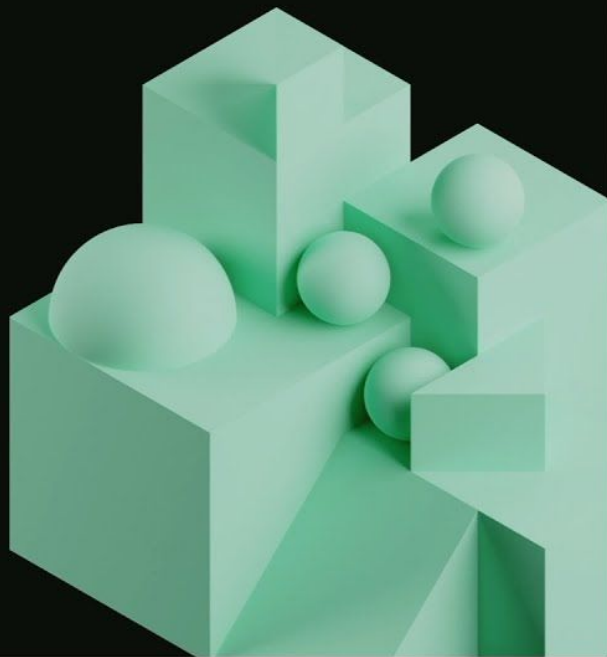
[Youtube video](#)

**DATA+AI
SUMMIT**
BY  databricks

MLOps at Gucci: from zero to hero

An overview on implementing an MLOps solution from scratch


Databricks
2023



MLOps at academia

It just doesn't exist (unfortunately)

You publish or perish:
no time for bullshit



What about open source stack?

Will discuss it during all the course.

Main problem of FOSS instruments - **lack of orchestration**.

Each instrument primarily aims to solve one task but then start to add some default solutions for adjacent problems which are not that great.

All in all no instrument solve every problem.

Finally you need to **write glue code yourself**.

Homeworks

girafe
ai

05

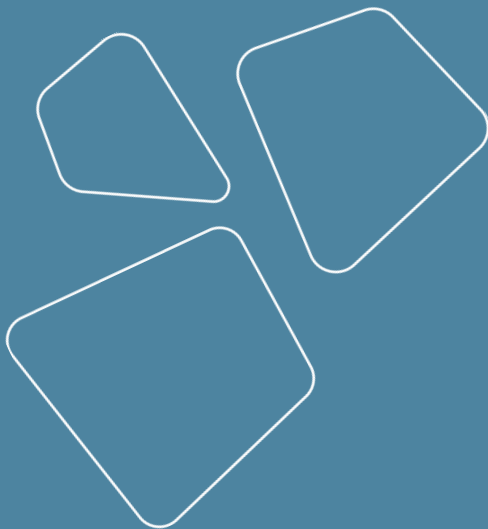
Homeworks

1. Formulate problem to apply MLOps stack
2. Write baseline code
3. Apply parts of pipeline from classes
4. Peer review
5. Final review

Also we will have simple tests for each lecture

Outline

- Motivation
- What is MLOps?
- Subtasks
 - and course structure
- Stacks overview
- Homeworks outline



Python toolbox

girafe
ai

05

Modules and packaging

- [python package structure](#)
 - common extensible format
 - able to be packaged
- build systems

[More on python packaging](#)

sound/	Top-level package
__init__.py	Initialize the sound package
formats/	Subpackage for file format conversions
__init__.py	
wavread.py	
wavwrite.py	
aiffread.py	
aiffwrite.py	
auread.py	
auwrite.py	
...	
effects/	Subpackage for sound effects
__init__.py	
echo.py	
surround.py	
reverse.py	
...	
filters/	Subpackage for filters
__init__.py	
equalizer.py	
vocoder.py	
karaoke.py	
...	



pyproject.toml

Standard file format for modern package metadata

[Packaging user guide](#), [Pip documentation on pyproject](#)

[Set of tools supporting pyproject.toml](#)

```
[project]
name = "mypackage"
version = "2023.05.1"
description = "A package for demonstrating Python packaging"

[build-system]
requires = ["setuptools >= 61.0.0"]
build-backend = "setuptools.build_meta"

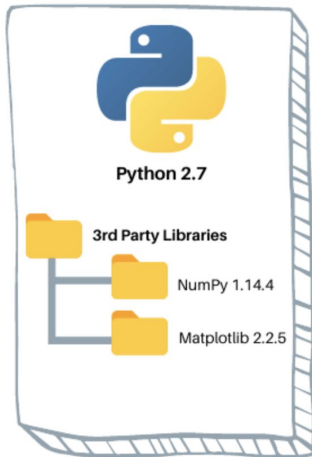
[tool.setuptools.packages.find]
where = ["src"]
```



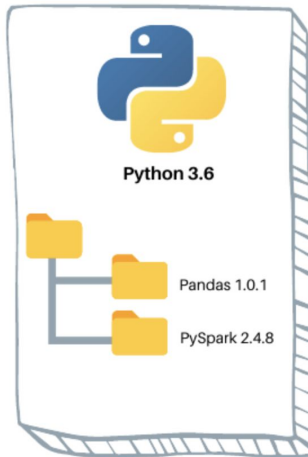
What do we want from environments?

- Create environments
 - separate for each project on one host machine
- Remember and install packages
 - repeatable set of packages for each project

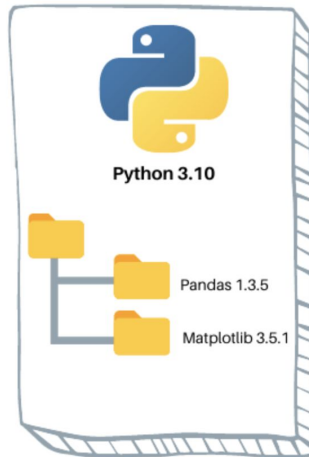
Virtual Environment 1



Virtual Environment 2



Virtual Environment 3



Vanilla solution

- Create environments
 - `python -m venv /path/to/new/virtual/environment`
- Remember and install packages
 - `pip freeze > requirements.txt`
 - `pip install -r requirements.txt`

Any problems with this?

- python version not saved
- only python dependencies are managed
- long and structureless requirements file (no dependencies)
- no groups of dependencies (prod vs dev)
- not portable to all OSes



Managing environments

- [venv](#) - Python standard library
- [virtualenv](#) - extended toolkit for virtual environments
- [poetry](#)
- conda - able to install non python dependencies
 - [miniconda](#)
 - [mamba](#)
- [pyenv](#) - automatic envs switching ([good article](#))
- [pipx](#)
- [pipenv](#)

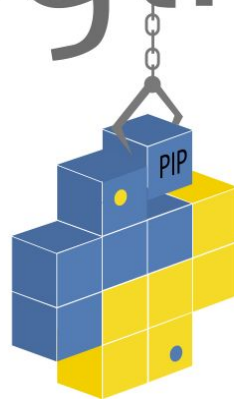
[More on virtual envs](#)



Managing dependencies

- pip (via requirements.txt)
- [poetry](#) - convenient tool covering most needs
- [pip-tools](#)
- [Pipfile](#) (via pipenv)
- conda
 - `conda env export > environment.yml`

python



Build systems

- setuptools
- poetry
- [Hatchling](#)
- flit
- ...



Poetry capabilities

- dares to install ONE tool per OS
 - also available for CI installation
- manages environments (optional)
- saves Python version
- resolves conflicts better due to hierarchical dependency
- supports groups of dependencies
- manages dependencies for all platforms simultaneously
- additional package indexes made easy

<https://python-poetry.org/docs/>

[Nice tutorial on Poetry packaging](#)



Poetry usage

Creates two files:

- `pyproject.toml` - initial dependencies, [config for all tools](#)
- `poetry.lock` - all concrete dependencies (platform agnostic)

If using conda environments:

```
poetry config virtualenvs.create false
```

For installing dependencies:

```
poetry install
```

Also it is able to build and publish to PyPI or your own package registry

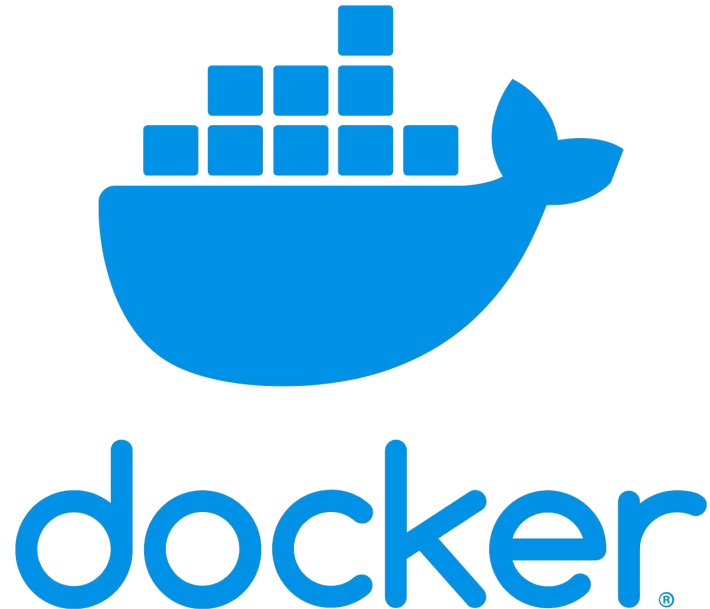
```
poetry build && poetry publish
```



Docker

As environment for Python:

- No change of OS
- No change of Python
- Dependencies compiled into image
- Pure pip can be used



Package repositories

- PyPI
 - can mirror public version
- custom
 - Artifactory
 - etc.



pypi



Command Line Interfaces (CLI)

Default tool for CLI in Python is [argparse](#)

Which is clone of C library and is painful to use

```
import argparse

parser = argparse.ArgumentParser(description='Process some integers.')
parser.add_argument('integers', metavar='N', type=int, nargs='+',
                    help='an integer for the accumulator')
parser.add_argument('--sum', dest='accumulate', action='store_const',
                    const=sum, default=max,
                    help='sum the integers (default: find the max)')

args = parser.parse_args()
print(args.accumulate(args.integers))
```



CLIs for humans

Also there are several tools made for humans.

One of them is [Fire](#).

It makes CLI out of pure Python functions and classes.

Other options:

<https://click.palletsprojects.com/>

```
import fire

def hello(name="World"):
    return "Hello %s!" % name

if __name__ == '__main__':
    fire.Fire(hello)
```

Then, from the command line, you can run:

```
python hello.py # Hello World!
python hello.py --name=David # Hello David!
python hello.py --help # Shows usage information.
```



Changelog

Changelog generation:

<https://towncrier.readthedocs.io/en/stable/>



Reference projects

1. Template project with basic setup
<https://github.com/v-goncharenko/data-science-template>
2. Good example of project with most of tools applied
<https://github.com/v-goncharenko/mimics/>
3. <https://github.com/NeuroTechX/moabb>



How to write a code

We will talk about it next time, now some great references:

- Summary of the Clean Code book
<https://gist.github.com/wojteklu/73c6914cc446146b8b533c0988cf8d29>
- Same thing adopted for Python
<https://github.com/zedr/clean-code-python>
- And for ML
<https://github.com/davified/clean-code-ml>

There's no silver bullet in code writing,
so despite all the books you need to think yourself.



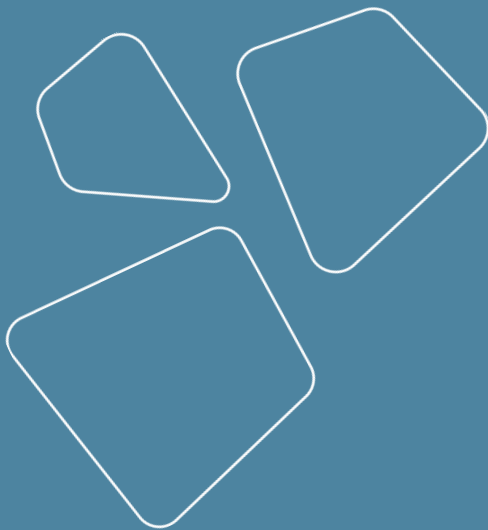
Homeworks

1. Formulate
2. Write base repository
3. Add tracking
4. Create trained model artifact
5. Write wrapper service
6. Deploy it
7. Annotate data?



Revise

- Motivation
- What is MLOps?
- Subtasks
- and course structure
- Stacks overview
- Homeworks outline



Thanks for attention!

Questions?

